

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)

Assessment Tool Weightage: 30%

Dr Dumpala Prasanth

QUESTIONS: 10

Total Marks:50

Annexure A

Constraint - Based Problem-Solving

Duration: 2Hrs

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.
(10 Marks)
2. Implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ϵ values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.
(10 Marks)
3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.
(10 Marks)
4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.
(10 Marks)

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

(10 Marks)

6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.

(10 Marks)

7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization.

(10 Marks)

8. Analyse an experimental comparison between random action selection and ϵ -greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

(10 Marks)

9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process.

(10 Marks)

10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

(10 Marks)

Code of Conduct:

I V. Sunil Kumar / 192425292 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

V. Sunil Kumar
Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Name: V. Sunil Kumar

Reg. No: 192425292

Code: MLA0304

- ① Developed a Python-based Reinforcement Learning Model using the openAI Gym environment to solve a Grid world navigation problem.

Reinforcement Learning (RL) is a machine learning technique where an agent learns by interacting with an environment.

State Space:

* Represents all possible positions of the agent.

* Example: In a 5x5 grid, there are 25 states.

Action Space:

* UP * Down * Left * Right

Reward Function:

→ Goal = +10

→ obstacle = -10

→ Normal move = -1

→ Invalid move = -2

Constraints:

Cannot move outside the grid.

cannot enter obstacle cells.

Episode ends after reaching the goal.

Python code:

```
import gym
```

```
env = gym.make("FrozenLake-v1", is_slippery=False)
```

```
state = env.reset()
```

```
done = False
```

```
while not done:
```

```
    action = env.action_space.sample()
```

```
    print(state, reward)
```

```
env.close()
```

CONCLUSION: The RL Agent learns the shortest and safest path by maximizing rewards and avoiding obstacles.

- ② Implement an ϵ -Greedy Multi-Armed Bandit Algorithm
The greedy algorithm balances exploration and exploitation
The experiment runs for 500 trials.

Python code:

```
import numpy as np
```

```
arms = [1.2, 2.5, 3.2, 2.0]
```

```
trials = 500
```

```
for epsilon in [0.1, 0.3, 0.5]:
```

```
    values = np.zeros(4)
```

```
    count = np.zeros(4)
```

```
    for i in range(trials):
```

```
        if np.random.rand() < epsilon:
```

```
            arm = np.random.randint(4)
```

```
        else:
```

```
            arm = np.argmax(values)
```

```
        reward = np.random.normal(arms[arm], 1)
```

```
        count[arm] += 1
```

```
        values[arm] += (reward - values[arm]) / count[arm]
```

```
    print("Epsilon:", epsilon)
```

```
    print(values)
```

Comparison:

ϵ value	Exploration	Exploitation	Result
0.1	Low	High	Highest reward
0.3	Medium	Medium	Balanced
0.5	High	Low	More exploration

CONCLUSION: Lower ϵ gives better exploitation, while higher ϵ encourages exploration.

③ Design a Markov Decision process (MDP) for an Autonomous Robot.

A Markov Decision process (MDP) models decision-making where outcomes depends on the current state and selected action.

States:

→ Robot Positions

→ Battery level

Actions:

* Move up

* Move Down

* Move Left

* Move Right.

Transition probabilities:

→ Correct move = 0.9

→ Incorrect move = 0.1

Rewards:

→ Destination = +100

→ Normal Move = -1

→ Obstacle = -20

Energy constraint

→ Battery = 100 units

→ Each movement consumes 1 unit.

CONCLUSION: The MDP helps the robot select actions that minimize energy consumption while reaching the destination.

Q Implement a Value Iteration Algorithm using Python and explain the Bellman Equation.

Value Iteration is a reinforcement learning algorithm used to find the optimal policy by repeatedly updating the value of each state.

Python code:

```
import numpy as np
```

```
gamma = 0.9
```

```
V = np.zeros(5)
```

```
reward = [0, -1, -1, 0, 20]
```

```
for i in range(20):
```

```
    for s in range(4):
```

```
        V[s] = reward[s] + gamma * V[s+1]
```

```
print(V)
```

Bellman Equation:

$$V(s) = \max_a \{R(s, a) + \gamma \sum P(s'|s, a) V(s')\}$$

Where:

$V(s)$ = value of state

$R(s, a)$ = Immediate reward

γ = Discount factor

$P(s'|s, a)$ = Transition probability.

CONCLUSION: Bellman updates continue until the optimal value function and policy are obtained.

⑤ Design an RL Framework for a warehouse Robot with a 30-Minute Battery.

The warehouse robot must complete the maximum number of deliveries without exceeding its 30-min battery limit.

States:

- Robot location
- Battery level
- Delivery status

Actions:

- * Pickup package
- * Deliver package
- * Recharge
- * wait.

Reward Design

- Successful delivery = +20
- Low battery = -10
- Idle time = -2
- Recharge = +5

Constraints:

- * Battery capacity is limited to 30 minutes.
- * Recharge before the battery is fully depleted
- * Avoid unnecessary movement.

Policy:

The robot selects the shortest delay route.

CONCLUSION: The RL framework enabled the robot to maximize deliveries while satisfying battery constraints through efficient policy learning.

⑥ Implement a Reinforcement learning agent that controls traffic signals while giving priority to emergency vehicles.

A Reinforcement learning (RL) agent can control traffic signals by learning the best timing for green and red lights.

States:

- Number of vehicles in each lane
- Traffic signal status.
- Presence of an emergency vehicle.

Action:

- * Green signal for North-South
- * Green signal for East-West
- * Extend green time
- * Switch signal.

Reward function:

- Smooth traffic flow = +10
- Reduced waiting time = +5
- Traffic congestion = -5
- Delay to emergency vehicle = -20

Python code:

```
emergency = True
```

```
if emergency:
```

```
    signal = "GREEN"
```

```
else:
```

```
    signal = "NORMAL RL ACTION"
```

```
print("Traffic signal:", signal)
```

CONCLUSION: The RL agent improves traffic flow while ensuring emergency vehicles receive immediate priority through safety constraints.

⑦ Develop a Python program using TensorFlow/Keras for packet routing in a wireless network with limited bandwidth.

In Packet Routing, the RL agent selects the best path for sending packets through a wireless network. The objective is to maximize successful packet delivery while minimizing congestion caused by limited bandwidth.

Python code:

```
import tensorflow as tf
Model = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dense(32, 3)
])
Model.compile(optimizer="adam", loss="mse")
print("RL Model created")
```

Effect of Bandwidth constraint:

Limited bandwidth reduces the search for congested routes. The agent learns to select routes with lower congestion and better packet delivery.

States:

- Current router
- Available bandwidth
- Network congestion

Actions:

- * Select Route A
- * Select Route B
- * Select Route C

CONCLUSION: The RL Model improves routing efficiency by considering bandwidth limitations during policy learning.

③ Compare Random Action selection and ϵ -Greedy Action selection in a Multi-Armed Bandit Problem.

Random action selection chooses actions without considering previous rewards, while the ϵ -greedy method balances exploration and exploitation to maximize cumulative rewards.

Python code:

```
import random
random-reward = 0
greedy-reward = 0
for i in range(500)
    random-reward += random.randint(0,5)
    if random.random() < 0.1:
        greedy-reward += random.randint(0,5)
    else:
        greedy-reward += 5.
Print("Random Reward:", random-reward)
Print("Epsilon-Greedy Reward:", greedy-reward)
```

Comparison:

Method	Exploration	Exploitation	Performance
Random	High	None	Lower reward
ϵ -Greedy	Balanced	High	Higher reward

Conclusion: The ϵ -greedy strategy achieves higher cumulative rewards because it balances exploration with exploitation, making it more effective than random action selection.

⑨ Reinforcement Learning Framework for a Delivery Drone

To design an RL framework that helps a delivery drone reach its destination while avoiding no-fly zones and maintaining battery limits.

MDP Formulation:

- States: Drone Position and battery level
- Actions (A): Move up, Down, Left, Right.
- Transition (P): Probability of Moving to the next state after an action.
- Reward (R): Goal reached: +100.
Normal Move: -1
- Constraint: Battery must not exceed its limit and no fly zones must be avoided.

Bellman equation:

$$V(S) = \max_a [R(S,a) + \gamma V(S')] \quad ;$$

Python Program:

```
Import numpy as np
```

```
Q = np.zeros((4,4))
```

```
Print ("Initial Q-Table")
```

```
Print(Q)
```

```
Print ("Training completed")
```

Result: The drone learns the safest route while avoiding restricted areas and conserving battery power.

⑩ Reinforcement Learning for Smart Energy Management.

To develop an RL system that keeps electricity consumption below a specified threshold.

State Space:

Current electricity consumption.

Action Space:

- * Turn ON appliance
- * Turn OFF appliance
- * Energy saving mode

Reward Function:

- * Below threshold: +20
- * Energy-saving action: +10
- * Above threshold: -50

Python Program:

```
Import numpy as np
```

```
Q = np.zeros((5,2))
```

```
print('Initial Q-table')
```

```
Print(Q)
```

```
Print("Training completed")
```

CONCLUSION: Reinforcement Learning improves the energy management by selecting actions that minimize electricity usage while satisfying system constraints.